

Chapter 1

INTRODUCTION

1.1 BACKGROUND

In educational institutions, students and visitors frequently require information related to departments, courses, faculty details, office locations, and academic notices. Traditionally, such information is obtained through notice boards, administrative offices, or college websites. These methods often lead to delays, repeated queries to staff, and difficulty in accessing accurate information at the right time.

With the increasing use of digital platforms and conversational interfaces, chatbots have emerged as an effective solution for automating information retrieval. Chatbots provide instant responses, operate continuously, and reduce the need for manual assistance. In a college environment, a chatbot can serve as a centralized information system that improves accessibility and efficiency.

The COET Chatbot is developed to address these challenges by providing a user-friendly, web-based conversational system for the College of Engineering Thalassery. It enables users to obtain essential college-related information through natural language queries, thereby minimizing dependency on administrative staff and enhancing the overall information delivery process.

1.2 OBJECTIVE

The main objective of the COET Chatbot is to provide an automated and user-friendly system for answering common academic and administrative queries related to the College of Engineering Thalassery.

The specific objectives of the project are:

- To develop a web-based chatbot that provides instant responses to college-related queries.
- To allow users to access information about departments, courses, HODs, and office details without manual assistance.
- To reduce the workload of administrative staff by automating repetitive queries.
- To provide an admin module for managing departments, FAQs, and chatbot responses.
- To ensure accuracy and consistency of information through a structured database.
- To improve accessibility and user experience by offering a simple conversational interface.

1.3 PROJECT DESCRIPTION

The COET Chatbot is a web-based conversational application developed to provide instant academic and administrative information related to the College of Engineering Thalassery. The system enables users to interact with the chatbot using natural language queries and receive accurate responses without the need for manual assistance or authentication.

The project consists of two main components: the User Interface Module and the Admin Module. The User Interface Module allows students, faculty, and visitors to ask questions related to departments, courses, Heads of Departments (HODs), office contact details, and frequently asked questions. The chatbot processes user input and retrieves appropriate responses from a structured backend database.

The Admin Module provides a secure interface for authorized administrators to manage departments, FAQs, keywords, and related information. Administrators can add, update, or delete data, ensuring that the chatbot's knowledge base remains current and reliable. This modular design makes the system easy to maintain and scalable for future enhancements.

The COET Chatbot improves information accessibility, reduces administrative workload, and enhances user experience through automation. The project demonstrates the effective use of web technologies and basic natural language processing concepts to build a practical and efficient college information system.

Chapter 2

SYSTEM OVERVIEW

2.1 Existing System

In the existing system, information related to academic and administrative activities in the College of Engineering Thalassery is primarily provided through college notice boards, official websites, printed circulars, and direct interaction with administrative staff. Students and visitors often need to visit different offices or departments to obtain basic information such as course details, department contacts, and faculty information.

This manual approach has several limitations. Information may not always be updated in real time, leading to confusion and repeated queries. Administrative staff are required to handle the same questions repeatedly, increasing their workload and reducing efficiency. Additionally, access to information is restricted to office working hours, making it inconvenient for users who require information outside these hours.

Due to the lack of an automated and centralized query-handling system, users may experience delays in receiving responses, and the overall information retrieval process becomes time-consuming. These challenges highlight the need for an intelligent, automated solution to improve accessibility, accuracy, and efficiency in information delivery.

2.2 Proposed System

The proposed system introduces the **COET Chatbot**, an automated, web-based conversational platform designed to provide instant academic and administrative information related to the College of Engineering Thalassery. The system acts as a centralized information source that allows users to interact with the chatbot using natural language queries and receive accurate responses in real time.

Unlike the existing manual system, the proposed chatbot operates continuously and does not depend on office working hours. Users can access information such as department details, courses, Heads of Departments (HODs), office contact information, and frequently asked questions through a simple chat interface without requiring login.

The system includes a secure Admin Module that enables authorized administrators to manage departments, FAQs, and keywords efficiently. Any updates made through the admin panel are immediately reflected in the chatbot responses, ensuring that the information remains current and reliable.

By automating query handling, the proposed system reduces administrative workload, minimizes response time, and improves overall accessibility to institutional information. The modular and scalable design of the COET Chatbot allows for future enhancements such as advanced natural language processing, voice-based interaction, and integration with institutional databases.

2.3 MODULE DESCRIPTION

The **COET Chatbot** system is divided into **two main modules: the User Interface Module and the Admin Module**. Each module is designed to perform specific functions and together they ensure efficient operation of the chatbot system.

2.3.1 USER INTERFACE MODULE

The **User Interface Module** provides an interactive chat-based interface through which students, faculty, and visitors can interact with the **COET Chatbot**. This module allows users to submit queries in natural language and receive instant responses without the need for authentication.

The chatbot processes user input and retrieves relevant information from the backend database. It provides answers related to departments, courses, Heads of Departments (HODs), office contact details, and frequently asked questions. The interface is designed to be simple, responsive, and user-friendly to ensure easy accessibility for all users.

Functions of the **User Interface Module**:

- Accepts natural language queries from users
- Displays instant chatbot responses
- Provides college, department, and course-related information
- Does not require user login
- Enhances user experience through a simple chat interface

2.3.2 ADMIN MODULE

The **Admin Module** is a secure management interface designed for authorized personnel to maintain and update the chatbot's knowledge base. This module ensures that all information provided by the chatbot remains accurate, relevant, and up to date.

Administrators can manage departments, HOD details, office contact information, and frequently asked questions through the admin panel. The module also supports keyword and tag management to improve the accuracy of chatbot responses.

Functions of the Admin Module:

- Secure admin login and logout
- Add, update, and delete department details
- Manage FAQs and chatbot responses
- Maintain contact and office information
- Update keywords for improved query matching
- Ensure data accuracy and consistency

Chapter 3

METHODOLOGY

3.1 AGILE SOFTWARE DEVELOPMENT

Agile Software Development is an iterative and incremental methodology that emphasizes flexibility, continuous user involvement, and rapid delivery of functional software. Unlike traditional rigid development models, Agile promotes developing software in small, manageable increments, enabling teams to adapt to changing requirements throughout the project lifecycle.

For the **COET Chatbot**, the Agile approach is particularly suitable as the system is designed to evolve based on user interactions and institutional requirements. As students, faculty, and visitors interact with the chatbot, new information needs and improvements emerge, influencing feature priorities such as FAQ handling, department information accuracy, and response quality. Agile allows the development process to accommodate these changes efficiently without disrupting the overall project structure.

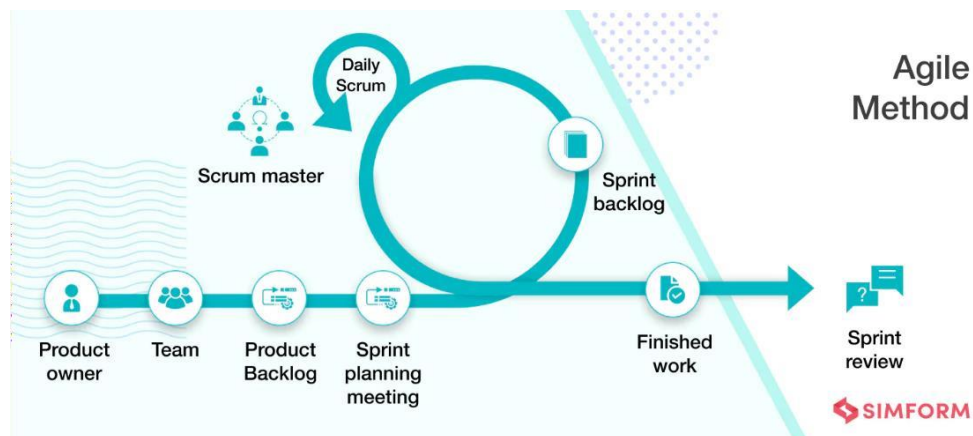
Agile also contributes significantly to risk reduction by encouraging early development, testing, and validation of individual system components. In the COET Chatbot project, core features such as the user chat interface, query processing logic, and admin management module were developed and tested incrementally. This approach helped identify and resolve technical issues at early stages, ensuring system stability before introducing additional features.

The methodology supports frequent delivery of working software, enabling the COET Chatbot to be developed and validated in stages. Initial iterations focused on implementing basic chatbot responses and static information retrieval, followed by enhancements such as dynamic FAQ management and improved keyword-based query matching. This incremental delivery ensured early validation of essential functionalities while allowing

continuous refinement of the user interface and chatbot behavior based on feedback.

Another important aspect of Agile applied in the COET Chatbot project is its emphasis on stakeholder collaboration. Feedback from students and faculty was considered to improve response relevance, usability, and information organization. Agile also promotes adaptability by allowing development priorities to shift based on actual usage patterns and administrative requirements, thereby ensuring that the chatbot remains effective, user-friendly, and aligned with institutional needs.

Overall, the Agile methodology, supported by short development iterations, continuous backlog refinement, clearly defined acceptance criteria, and a structured yet flexible workflow, provided a reliable framework for the successful development of the **COET Chatbot mini project**.



3.2 SCRUM FRAMEWORK

The Scrum framework is implemented in this project to facilitate team-based development and to streamline the Agile process. Scrum provides a structured approach for managing iterative development through clearly defined roles, events, and artifacts, enabling efficient planning, development, and delivery of the system.

Scrum focuses on three primary roles that together form the foundation of the development process:

1. Product Owner (PO):

The Product Owner represents stakeholders and is responsible for defining and prioritizing the requirements of the COET Chatbot. The Product Owner maintains the product backlog by identifying key features such as user query handling, department information management, FAQ responses, and administrative functionalities. By prioritizing backlog items, the Product Owner ensures that the most valuable features are developed first.

2. Scrum Master:

The Scrum Master facilitates the Scrum process and ensures that Agile principles and Scrum practices are followed throughout the development lifecycle. In the COET Chatbot, the Scrum Master helps remove impediments that may affect progress, coordinates Scrum events, and supports the development team in maintaining an efficient and collaborative workflow.

3. Development Team:

The Development Team is a cross-functional and self-organizing group responsible for analysis, design, development, testing, and deployment of the COET Chatbot. The team collaborates closely to complete the planned work in each sprint and organizes itself to deliver a functional system increment at the end of every sprint.

The COET Chatbot was developed using the Scrum framework, with each sprint focusing on high-priority modules such as the user interaction interface, admin management features, chatbot response logic, and database integration. This sprint-based approach ensured structured, incremental development, continuous testing, and steady enhancement of system functionality.

KEY SPRINTS INCLUDED

1. User Interface & Admin Authentication Sprint

This sprint focused on designing and implementing the basic user interaction interface along with secure administrative access. It included the development of the chatbot user interface for query input and response display, as well as admin login functionality. The sprint also established role-based access to ensure that only authorized administrators could manage chatbot data. This sprint laid the foundation for secure system access and smooth interaction.

2. Department & Information Management Sprint

This sprint addressed the core information layer of the system. Features implemented included the creation and management of department details, Head of Department (HOD) information, office contact details, and basic academic information. The sprint ensured accurate storage, retrieval, and organization of institutional data required for responding to user queries.

3. FAQ & Query Processing Sprint

This sprint focused on implementing the chatbot's core query-handling functionality. It included keyword-based query matching, FAQ management, and response generation logic. The chatbot was enabled to process user queries efficiently and provide relevant responses based on stored data, improving accuracy and response time.

4. Admin Data Management & Update Sprint

This sprint introduced advanced admin functionalities for maintaining the chatbot's knowledge base. Administrators were provided with options to add, update, and delete FAQs, department details, and keywords. This sprint ensured that information presented by the chatbot remained current, consistent, and reliable.

5. Testing, UI Refinement & System Optimization Sprint

This sprint focused on overall system testing, user interface refinement, and performance

optimization. It included input validation, error handling, and improvements to chatbot response flow. Data consistency checks and usability enhancements were also carried out to ensure system reliability and improved user experience.

SCRUM FRAMEWORK IMPLEMENTATION

The use of the **Scrum framework** enabled the **COET Chatbot** to follow an adaptive and collaborative development workflow. Each sprint was time-boxed and designed to deliver a complete and testable set of chatbot features. Sprint planning sessions were conducted regularly to prioritize backlog items based on information relevance and user needs, while continuous progress tracking ensured steady development and quick resolution of issues.

Sprint review meetings were conducted at the end of each sprint to demonstrate completed functionalities such as chatbot responses, admin management features, and information retrieval workflows. Feedback obtained during these reviews was incorporated into subsequent sprints, supporting continuous improvement of the system. Sprint retrospectives played an important role in identifying workflow inefficiencies, improving task estimation, and refining development practices. The Scrum approach ensured clear communication, early testing, and timely delivery throughout the development lifecycle.

Several important refinements were identified and implemented through sprint retrospectives during the development of the COET Chatbot.

1. Chatbot Response Accuracy Improvements

Based on feedback, enhancements were made to improve the accuracy and relevance of chatbot responses. Keyword matching logic was refined, response prioritization was improved, and irrelevant or ambiguous responses were minimized, resulting in better user satisfaction.

2. Information Consistency & Data Validation Enhancements

Refinements were implemented to ensure consistency of department details, FAQs, and contact information. Improved input validation and structured data handling helped reduce

duplicate or incorrect entries, thereby increasing system reliability.

3. User Interface & Interaction Workflow Refinements

The chatbot interface and admin dashboards were refined to improve usability and navigation. Feedback led to simplified admin forms, clearer chatbot prompts, improved response formatting, and better visual organization of information, enhancing overall user experience.

3.3 USER STORY

User stories played a crucial role in guiding the development of the **COET Chatbot** by ensuring that each system feature directly addressed the practical needs of its users. Each user story clearly defines who requires a particular feature, what functionality is expected, and why it is important. This approach helped align system development with the information needs of students, faculty members, visitors, and administrators of the College of Engineering Thalassery.

By focusing on real-world usage scenarios, user stories enabled the development team to prioritize features such as accurate information retrieval, ease of interaction, and efficient administrative control. They also supported better communication between stakeholders and the development process, ensuring that chatbot responses were relevant, reliable, and user-friendly. Overall, the use of user stories contributed to a more structured and user-centric development of the COET Chatbot system.

Admin User Stories

- **As an admin**, I want to log in securely so that only authorized personnel can manage the chatbot system.
- **As an admin**, I want to add, update, and delete department details so that the chatbot provides accurate academic information.

- **As an admin**, I want to manage Head of Department (HOD) and office contact details so that users receive correct communication information.
- **As an admin**, I want to create and maintain Frequently Asked Questions (FAQs) so that common user queries are answered efficiently.
- **As an admin**, I want to manage keywords and responses so that the chatbot can match user queries accurately.
- **As an admin**, I want to update or remove outdated information so that data reliability and system integrity are maintained.
- **As an admin**, I want to review chatbot content periodically so that information remains consistent and up to date.

User Interface Module User Stories

- **As a user**, I want to interact with the chatbot through a simple chat interface so that I can easily ask questions.
- **As a user**, I want to get instant responses from the chatbot so that I can save time and obtain information quickly.
- **As a user**, I want to ask about department details so that I can understand academic offerings.
- **As a user**, I want to know the Head of Department and office contact details so that I can communicate with the appropriate department.
- **As a user**, I want to view frequently asked questions through the chatbot so that I can get quick answers to common queries.
- **As a user**, I want the chatbot to provide clear and accurate responses so that I can rely on the information given.
- **As a user**, I want the chatbot to be available at all times so that I can access information whenever needed.

3.4 SPRINT

Sprint planning played a significant role in the development of the **COET Chatbot** by enabling the work to be divided into clear, time-boxed development phases. Each sprint focused on implementing specific chatbot functionalities and addressing the information needs of students, faculty members, visitors, and administrators. This approach ensured continuous progress, early validation of features, and effective delivery of a reliable college information system.

Sprint Planning

During sprint planning, the development team collaboratively identified and prioritized features based on their importance to usability and information accuracy. High-priority functionalities such as the chatbot user interface, query processing logic, FAQ management, and admin data management were selected for implementation. User stories were decomposed into smaller development tasks and assigned appropriately for frontend design, backend logic, database management, and testing. Each task was estimated and included in the sprint backlog, with objectives clearly aligned to the roles of the **Admin Module** and the **User Interface Module**.

Sprint Execution and Review

Sprint execution involved the step-by-step development of core chatbot features including the interactive chat interface, keyword-based query matching, FAQ response handling, and administrative data management. Regular progress tracking helped monitor development status and resolve issues efficiently. At the end of each sprint, a sprint review session was conducted where completed functionalities were demonstrated and evaluated. Feedback received during these reviews was used to refine existing features and to plan enhancements for subsequent sprints.

Incremental Delivery

Each sprint resulted in a functional and testable increment of the **COET Chatbot**, allowing early feedback and validation. For example, after implementing basic chatbot responses,

refinements were made to improve response clarity, keyword matching accuracy, and information organization. This incremental delivery model enabled essential features to be developed early, reduced technical risks, and supported continuous improvement based on user and administrator feedback. As a result, the system evolved steadily while maintaining reliability, consistency, and user satisfaction.

Chapter 4

SYSTEM ANALYSIS AND DESIGN

4.1 DATABASE DESIGN

Table 1: AdminAccount

COLUMN NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION
id	BigInt	Primary Key, Auto-increment	Unique identifier for the admin.
username	Varchar(150)	Unique, Not Null	Admin login name.
password	Varchar(255)	Not Null	Hashed password string.
email	Varchar(254)	Optional	Admin contact email.
is_active	Boolean	Default: True	Status of the account.
created_at	DateTime	Not Null	Date of account

COLUMN NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION
			creation.

Table 2: User

COLUMN NAME	DATA TYPE	CONSTRAINT S	DESCRIPTIO N
id	BigInt	Primary Key, Auto-increment	Unique identifier for the user.
username	Varchar(150)	Unique, Not Null	Primary login username.
role	Varchar(20)	Choices, Default: 'student'	student, faculty, or admin.
department_id	BigInt	Foreign Key (Department)	Reference to the user's department.
tags	JSON	Default: []	List of searchable

COLUMN NAME	DATA TYPE	CONSTRAINT S	DESCRIPTIO N
			keywords.
search_text	Text	Optional	Combined text for search indexing.

Table 3: Department

COLUMN NAME	DATA TYPE	CONSTRAINT S	DESCRIPTIO N
id	BigInt	Primary Key, Auto-increment	Unique identifier.
code	Varchar(10)	Unique, Not Null	Short code (e.g., "CS101").
name	Varchar(200)	Not Null	Full department name.
hod_id	BigInt	Foreign Key	Reference to

COLUMN NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION
		(User)	the Head of Department.
location	Varchar(255)	Optional	Physical location on campus.
office_hours	Varchar(200)	Optional	Availability timing.

Table 4: FacultyProfile

COLUMN NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION
id	BigInt	Primary Key, Auto-increment	Unique identifier.
user_id	BigInt	One-to-One (User)	Link to the User account.

COLUMN NAME	DATA TYPE	CONSTRAIN TS	DESCRIPTI ON
designation	Varchar(200)	Optional	Job title (e.g., Professor).
department_id	BigInt	Foreign Key (Department)	Assigned department.
qualifications	Text	Optional	Educational background.
research_interests	Text	Optional	Areas of academic research.

Table 5: Course

COLUMN NAME	DATA TYPE	CONSTRAINT S	DESCRIPTIO N
id	BigInt	Primary Key, Auto-increment	Unique identifier.

COLUMN NAME	DATA TYPE	CONSTRAINT S	DESCRIPTIO N
code	Varchar(50)	Not Null	Course catalog code.
title	Varchar(255)	Not Null	Name of the course.
credits	Decimal(4,1)	Optional	Credit hour value.
department_id	BigInt	Foreign Key (Department)	Department offering the course.

Table 6: FAQ

COLUMN NAME	DATA TYPE	CONSTRAINT S	DESCRIPTIO N
id	BigInt	Primary Key, Auto-increment	Unique identifier.
question	Varchar(512)	Not Null	The question text.

COLUMN NAME	DATA TYPE	CONSTRAINT S	DESCRIPTIO N
answer	Text	Not Null	The detailed response.
department_id	BigInt	Foreign Key (Department)	Department relevant to the FAQ.
published	Boolean	Default: True	Visibility status.
created_by_id	BigInt	Foreign Key (User)	User who authored the FAQ.

Table 7: Notice

COLUMN NAME	DATA TYPE	CONSTRAINT S	DESCRIPTIO N
id	BigInt	Primary Key, Auto-increment	Unique identifier.
title	Varchar(255)	Not Null	Headline of the

COLUMN NAME	DATA TYPE	CONSTRAINT S	DESCRIPTIO N
	)		notice.
body	Text	Not Null	Main content of the notice.
publish_from	DateTime	Default: Now	Start date for visibility.
publish_to	DateTime	Optional	Expiry date for visibility.

Table 8: InfoPage

COLUMN NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION
id	BigInt	Primary Key, Auto-increment	Unique identifier.
slug	Slug	Unique, Not Null	URL-friendly identifier.
title	Varchar(255)	Not Null	Page title.

COLUMN NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION
content	Text	Not Null	Page body content.

Table 9: SuggestionPhrase

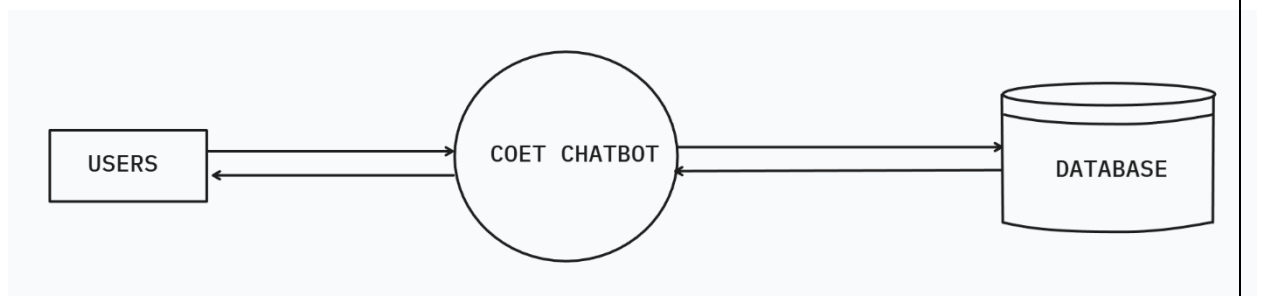
COLUMN NAME	DATA TYPE	CONSTRAINT S	DESSCRIPTIO N
id	BigInt	Primary Key, Auto- increment	Unique identifier.
phrase	Varchar(51 2)	Indexed, Not Null	The searchable string.
weight	Integer	Indexed, Default: 0	Priority for suggestion sorting.
content_type_i d	Integer	Foreign Key	Links to Django's ContentType.

COLUMN NAME	DATA TYPE	CONSTRAINT S	DESCRIPTIO N
object_id	Int Positive	Not Null	ID of the related object.

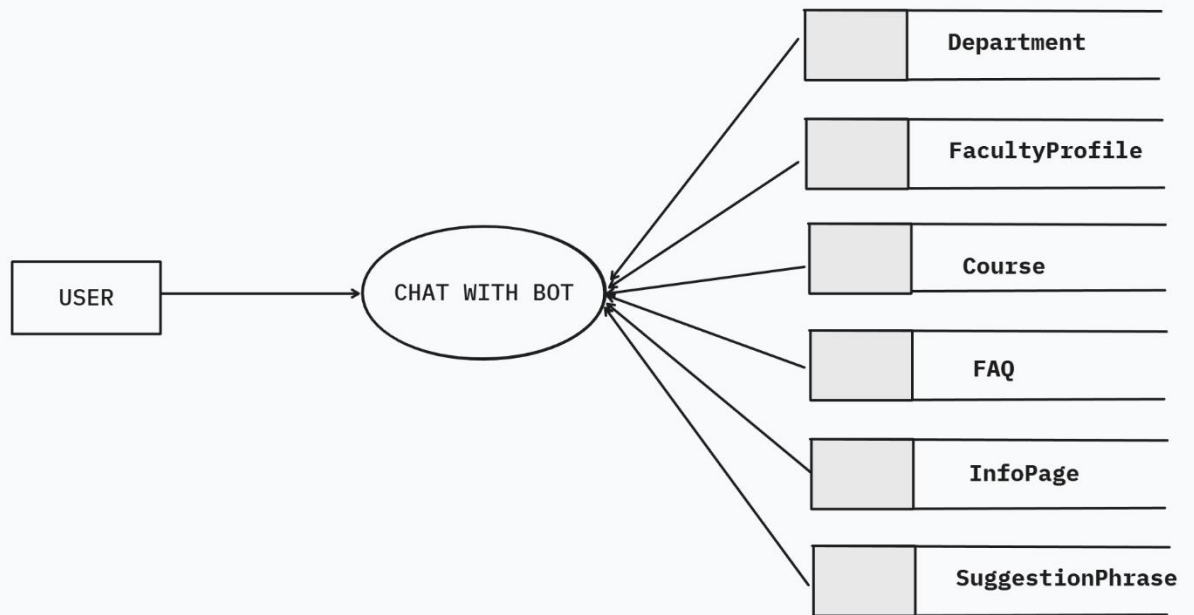
4.2 DATA FLOW DIAGRAM

Data flow diagrams are used to graphically represent the flow of data in a business information system. DFD describes the processes that are involved in a system to transfer data from the input to the file storage and reports generation.

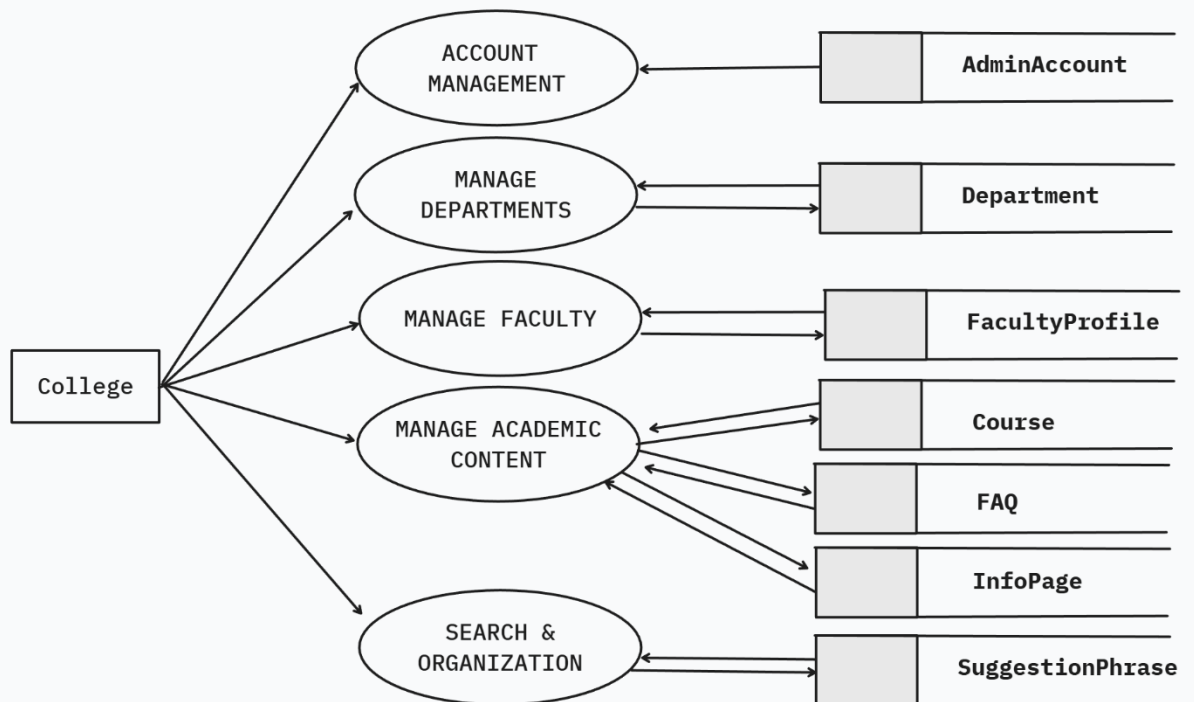
LEVEL 0



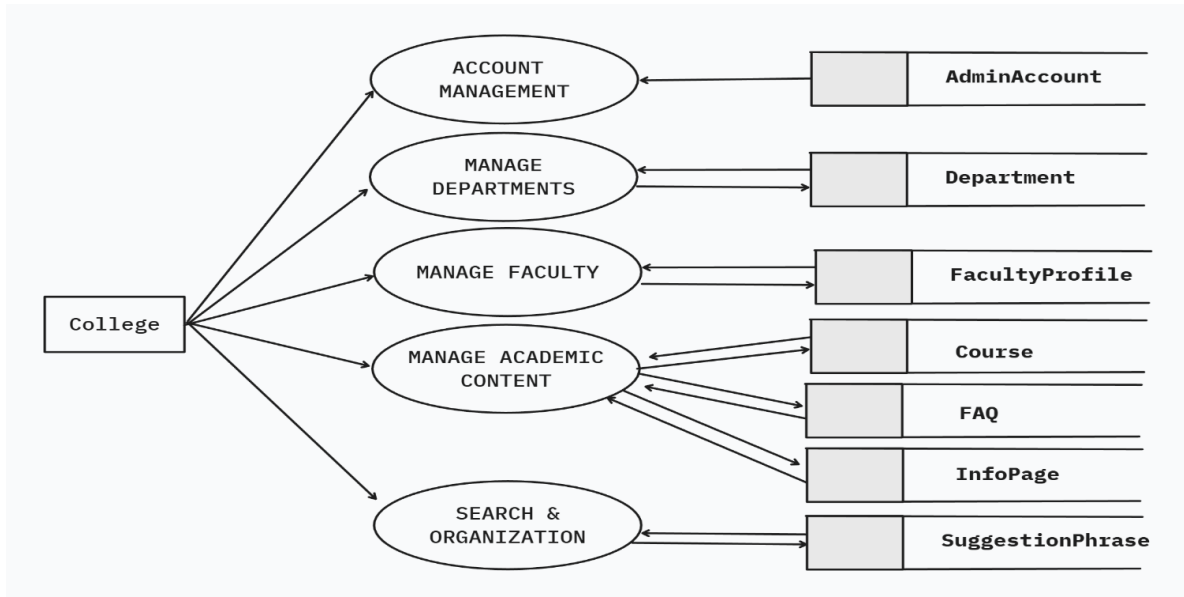
LEVEL 1 – USER



LEVEL 1 – CHATBOT

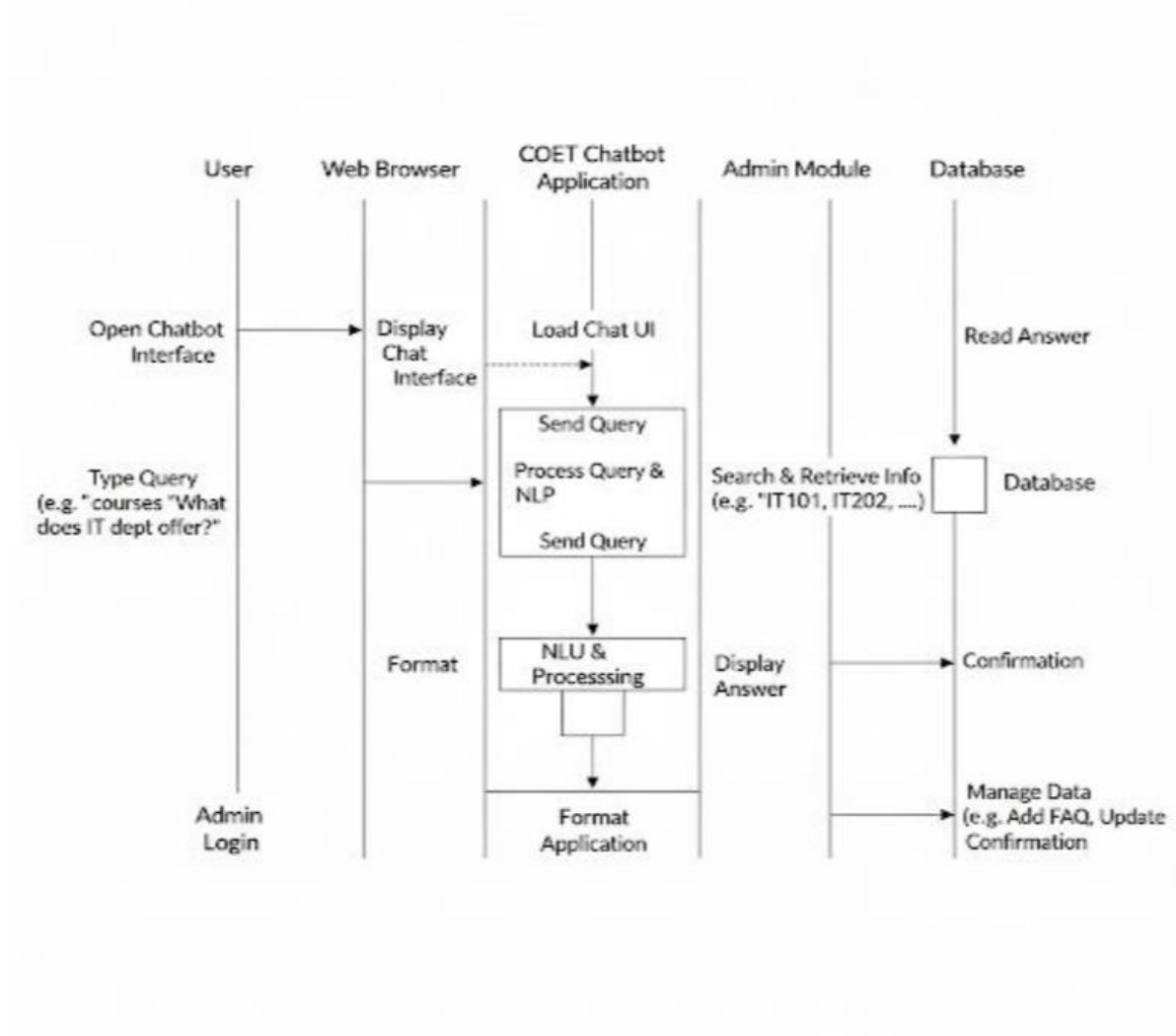


LEVEL 1 – COLLEGE



4.3 SEQUENCE DIAGRAM

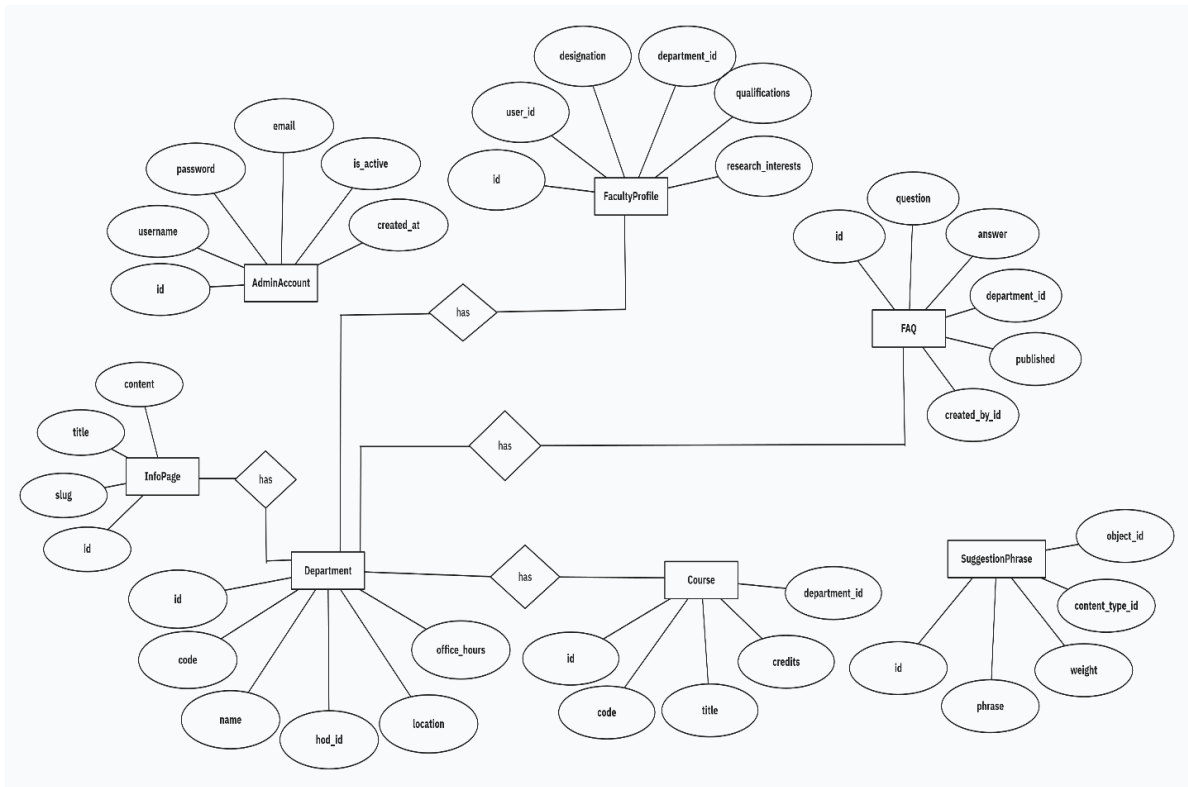
A Sequence Diagram is a Unified Modeling Language (UML) interaction diagram that maps the chronological flow of messages between objects or components required to complete a specific system function. It illustrates how different entities such as actors, interfaces, and databases collaborate over time, with the horizontal axis representing the participants and the vertical axis representing the passage of time. By detailing the exact order of requests and responses, it serves as a technical blueprint for the system's dynamic behavior, ensuring that the logic of a use case is consistent and that all internal communications are clearly defined for implementation.



4.4 ER DIAGRAM

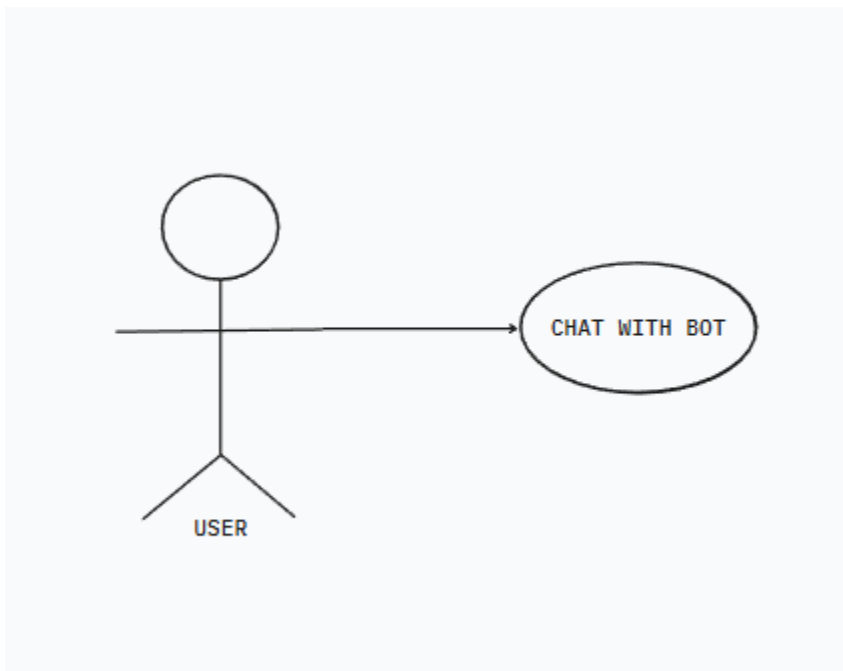
An **Entity–Relationship (ER) Diagram** is a graphical representation used in database design to describe the structure of a database. It visually shows the **entities** (objects or concepts about which data is stored), their **attributes** (properties of the entities), and the **relationships** between them.

ER diagrams help in understanding how data is organized and how different data items are connected before implementing the actual database. By clearly defining entities and their relationships, an ER diagram ensures proper data organization, reduces redundancy, and maintains data integrity. It acts as a blueprint for creating an efficient and well-structured database system.

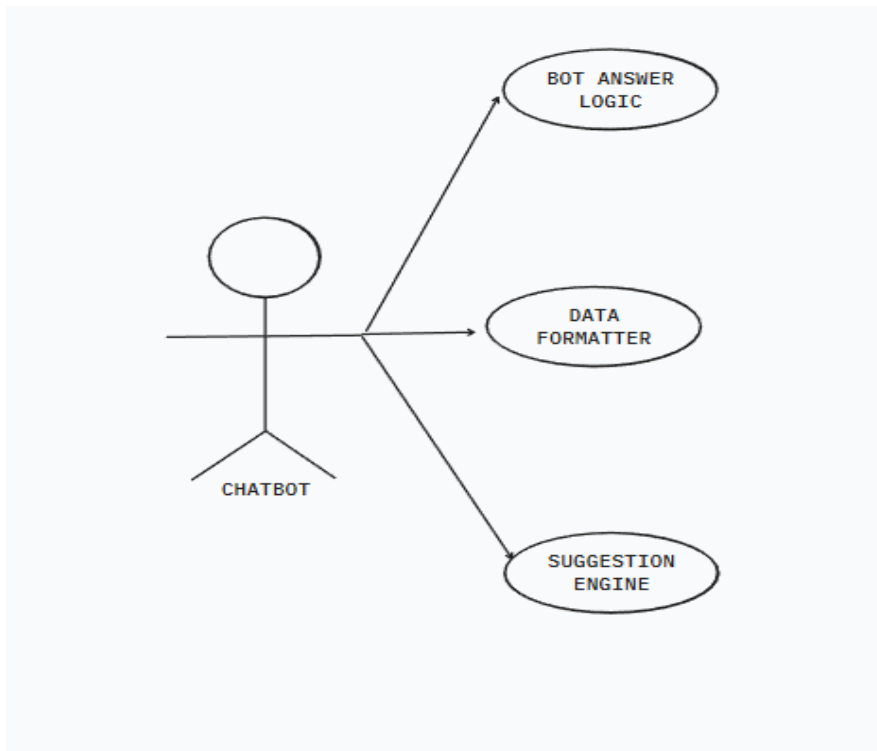


4.5 USE CASE DIAGRAM

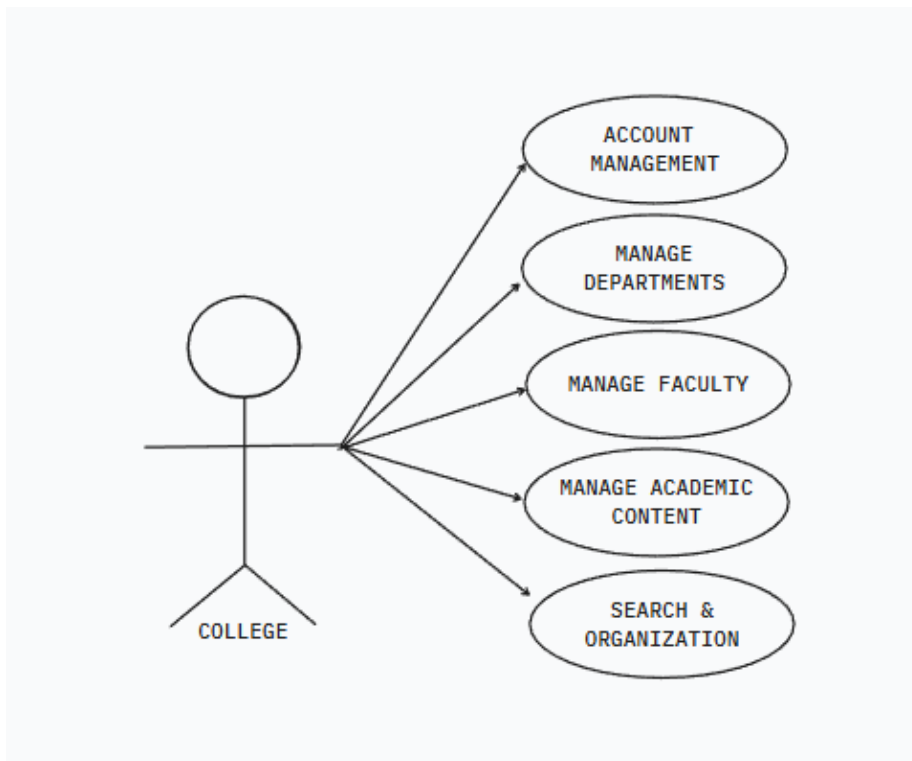
USE CASE FOR USER



USE CASE FOR CHATBOT



USE CASE FOR COLLEGE



Chapter 5

TOOLS AND PLATFORM

The development of the **COET Chatbot** utilizes modern, lightweight, and scalable technologies suitable for building efficient web-based information systems. The chosen tools support rapid development, easy maintenance, and reliable performance, making them ideal for an academic chatbot application.

The backend of the system is developed using the **Fast API** framework, which is known for its high performance, simplicity, and support for asynchronous operations. FastAPI is built on Python and is well suited for API-driven applications such as chatbots and information retrieval systems.

The COET Chatbot incorporates the following key technologies:

- **Python**
- **Fast API**
- **HTML, CSS, and JavaScript**
- **SQLite3**
- **Git and GitHub**
- **Visual Studio Code**

Each of these tools plays an important role in ensuring the efficiency, scalability, and maintainability of the COET Chatbot system.

5.1 PYTHON

Python serves as the primary programming language for the backend development of the **COET Chatbot** due to its simplicity, readability, and extensive library support. Python's clean syntax enables faster development and reduces the likelihood of programming errors, making it suitable for rapid application development.

Python is widely used in web development and automation, with strong community support and comprehensive documentation. Its flexibility allows it to be used for

backend logic, API development, database interaction, and data processing.

In the COET Chatbot system, Python is used to implement core functionalities such as chatbot logic, keyword-based query processing, admin authentication, database operations, and system validation. Its versatility also allows easy future enhancements such as advanced natural language processing and intelligent response handling.

5.2 FASTAPI

Fast API is the web framework used for developing the backend of the **COET Chatbot**. It is a modern, high-performance framework designed for building APIs using Python and is based on standard Python type hints.

Fast API provides automatic data validation, serialization, and interactive API documentation using Open API and Swagger UI, which improves development efficiency and testing. Its asynchronous request handling ensures fast response times, making it suitable for chatbot-based applications that require quick interaction.

In the COET Chatbot, Fast API handles API communication between the frontend and backend, processes user queries, manages admin operations, and retrieves information from the database. Its modular architecture supports scalability and easy maintenance.

5.3 HTML, CSS, AND JAVASCRIPT

HTML, CSS, and JavaScript form the core frontend technologies of the **COET Chatbot** and are responsible for delivering a simple and interactive user interface.

HTML (Hypertext Markup Language) provides the basic structure of the chatbot interface, including input fields, message display areas, and buttons. CSS (Cascading Style Sheets) is used to style the interface, control layouts, apply colors and fonts, and ensure responsiveness across different devices.

JavaScript adds interactivity to the chatbot by handling user input, dynamically displaying chatbot responses, and enabling smooth communication with the backend APIs. Together, these technologies ensure a user-friendly and responsive

chatbot experience.

5.4 SQLITE3

SQLite3 is used as the relational database management system for the **COET Chatbot**. It is a lightweight, serverless, and self-contained database engine, making it suitable for small to medium-scale applications and academic projects.

SQLite3 stores the database in a single file, simplifying configuration and maintenance. It supports standard SQL queries and ensures data consistency through ACID compliance.

In the COET Chatbot, SQLite3 is used to store department details, FAQs, keywords, chatbot responses, and admin credentials. It integrates seamlessly with Python, allowing efficient database operations and reliable data management.

5.5 VERSION CONTROL: GIT AND GITHUB

Git is a distributed version control system used to track changes in the COET Chatbot source code throughout the development process. It enables efficient management of code versions, collaboration, and rollback to previous states when required.

GitHub is used as a remote repository for storing and managing the project code. It provides features such as version tracking, issue management, and code sharing, ensuring organized and secure development.

5.6 VISUAL STUDIO CODE

Visual Studio Code (VS Code) is used as the primary development environment for the COET Chatbot. It provides a lightweight yet powerful code editor with features such as syntax highlighting, debugging support, integrated terminal, and extension support.

VS Code enhances developer productivity by offering tools for Python development, frontend design, and Git integration, making it suitable for full-stack web application development.

Chapter 6

IMPLEMENTATION

The implementation phase of the **COET Chatbot** project involved transforming the system design, data models, and user requirements into a fully functional web-based chatbot application. The platform was developed using **Python** and **FastAPI** for the backend, with **HTML**, **CSS**, and **JavaScript** for the frontend, and **SQLite3** as the relational database. The implementation followed an **Agile and sprint-based development approach**, allowing features to be developed, tested, and refined incrementally.

The backend implementation focused on creating **RESTful APIs** for core functionalities such as user authentication, question handling, context management, and response generation. **SQLAlchemy ORM** was used to define database models and manage database interactions efficiently, ensuring data consistency and integrity. Authentication and authorization mechanisms were implemented to provide secure access to the chatbot system based on user roles such as **Admin** and **User**.

Frontend implementation involved designing **responsive user interfaces** that allow users to interact with the chatbot, submit queries, view responses, and provide feedback. **Client-side validations** and **dynamic content loading** were implemented using JavaScript to enhance user experience and reduce unnecessary server requests. Admin-specific interfaces were developed to manage users, monitor queries, and update chatbot knowledge.

Throughout the implementation phase, **continuous testing** was performed to verify correctness, handle edge cases, and improve system stability. **Version control using Git and GitHub** ensured organized code management and traceability of changes. By the end of this phase, all planned core features of the COET Chatbot were successfully implemented and integrated into a unified system.

Chapter 7

SAMPLE CODE

7.1 Show suggestion:

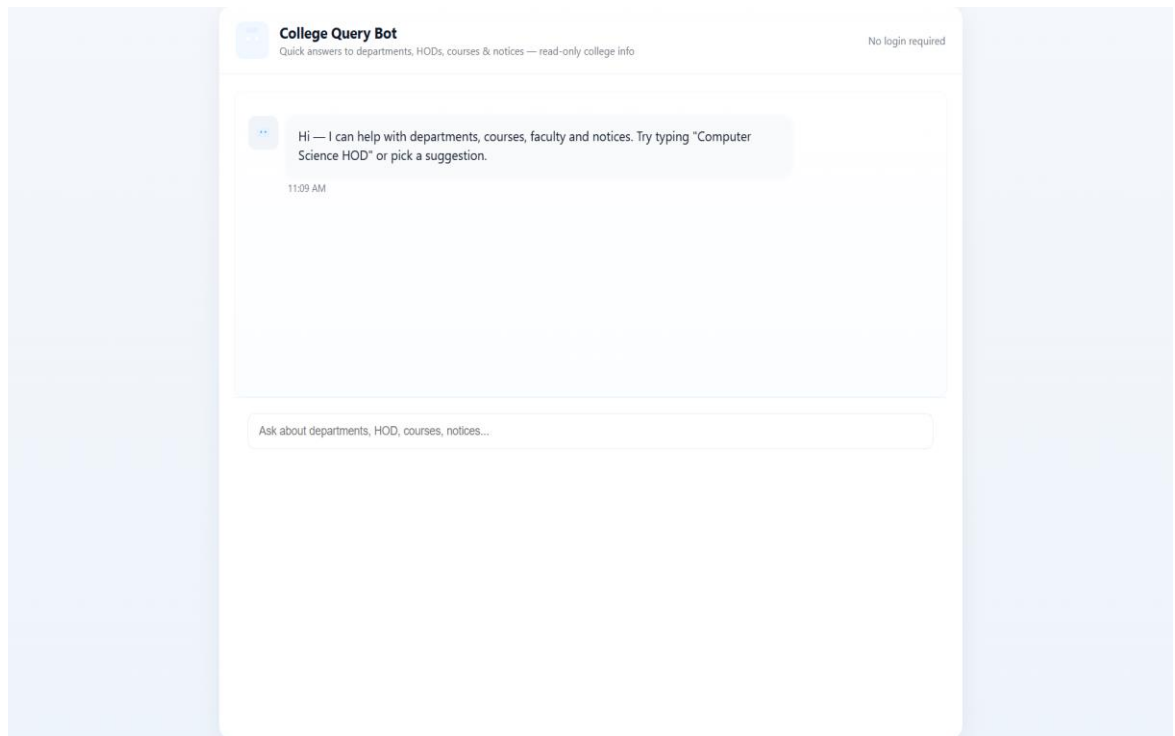
```
def suggestions_view(request):
    q = (request.GET.get('q') or '').strip()
    if not q:
        return JsonResponse({'suggestions': []})
    phrases =
    SuggestionPhrase.objects.filter(phrase__istartswith=q).order_by('-
weight')[:12]
    results = [{'phrase': p.phrase, 'object_type': p.content_type.model,
'object_id': p.object_id} for p in phrases]
    if len(results) < 12:
        extras = []
        models_to_search = [FAQ, InfoPage, Department, Course,
FacultyProfile]
        for m in models_to_search:
            qs = m.objects.filter(search_text__icontains=q)[:6]
            for obj in qs:
                phrase = (obj.search_phrases or [])
                phrase = phrase[0] if phrase else (getattr(obj, 'question',
None) or getattr(obj, 'title', None) or str(obj))
                extras.append({'phrase': phrase, 'object_type':
m._meta.model_name, 'object_id': obj.pk})
            if len(results) + len(extras) >= 12:
                break
            if len(results) + len(extras) >= 12:
                break
        results.extend(extras)
    return JsonResponse({'suggestions': results[:12]})
```

7.2 Dynamic object serializer:

```
def serialize_object(obj):
    if obj is None:
        return {'error': 'not found'}
    model = obj.__class__.__name__
    base = {'type': model, 'id': obj.pk}
    if isinstance(obj, FAQ):
        base.update({'title': obj.question, 'content': obj.answer,
'department': getattr(obj.department, 'name', None)})
    elif isinstance(obj, InfoPage):
        base.update({'title': obj.title, 'content': obj.content})
    elif isinstance(obj, Department):
        base.update({
            'title': obj.name,
            'code': obj.code,
            'office': obj.office,
            'phone': obj.phone,
            'email': obj.email,
            'location': obj.location,
            'website': obj.website,
            'office_hours': obj.office_hours,
            'hod': {
                'id': obj.hod.id if obj.hod else None,
                'name': obj.hod.get_full_name() if obj.hod else None,
                'email': obj.hod.email if obj.hod else None,
            }
        })
    elif isinstance(obj, Course):
        base.update({'title': obj.title, 'code': obj.code, 'description':
obj.description, 'department': getattr(obj.department, 'name', None)})
    elif isinstance(obj, FacultyProfile):
        base.update({
            'name': obj.user.get_full_name(),
            'designation': obj.designation, 'department':
getattr(obj.department, 'name', None),
            'phone': obj.phone, 'email': obj.email, 'bio': obj.bio,
            'research_interests': obj.research_interests
        })
    else:
        base.update({'repr': str(obj)})
    return base
```

CHAPTER 8

SCREENSHOTS



USER INTERFACE

College Admin Panel

Signed in as: admin

[Sign out](#)

Create Department

Code (e.g. CSE)

Department name

HOD username (optional)

Office

Phone

Email

Search phrases (comma-separated)

Tags (comma-separated)

Create Department

Create FAQ

Question

Answer

Department code (optional)

Search phrases (comma-separated)

Tags (comma-separated)

Create FAQ

Existing Departments

MBA — Master of Business Administration

Office: 4477555888 | Phone: 9988445566

MCA — MASTER OF COMPUTER APPLICATION

Office: 9988776655 | Phone: 889755446795

MECH — MECHANICAL ENGINEERING

Office: 9988776655 | Phone: 8877995544

Recent FAQs

What is the semester fees of MECH?

Dept: MECHANICAL ENGINEERING

What is the semester fees of MBA?

Dept: Master of Business Administration

what is the semester fees of MCA?

Dept: N/A

Fees?

Dept: N/A

Hi

Dept: N/A

Courses

ADMIN PAGE

39

Chapter 9

RESULTS AND CONCLUSION

7.1 Results and Conclusion

The COET Chatbot project successfully achieved its primary objective of developing an intelligent, interactive system to assist students and staff at the College of Engineering Thalassery (COET) with academic and administrative queries. The chatbot provides structured, easy-to-access information about courses, faculty, results, notices, and other college-related data, while improving efficiency and reducing response time for repetitive queries.

The implemented system allows users to obtain instant answers, receive guidance on procedures, and access relevant resources without manual intervention. Administrators can update content, manage frequently asked questions, and monitor chatbot interactions to ensure accuracy and relevance. The modular design and use of modern web technologies make the system scalable, maintainable, and adaptable for future enhancements.

Overall, the COET Chatbot demonstrates how automation and AI-driven interaction can enhance communication within a college environment. The project also validates the effectiveness of Agile development and sprint-based execution in delivering a functional, user-centric solution within a limited timeframe.

7.2 Scope of Future Work

Although the COET Chatbot currently provides essential query resolution and information retrieval, there is significant scope for future enhancements. One key improvement would be the integration of natural language understanding (NLU) models to interpret more complex, context-aware queries, enabling more human-like conversations.

Additional enhancements may include voice-based interaction, integration with

college databases for real-time updates (such as attendance, exam schedules, and notifications), and personalized recommendations for students based on their courses and interests. These improvements would make the chatbot an even more effective digital assistant, transforming it from a static query-answer system into a dynamic, intelligent support platform.

Chapter 10

REFERENCES

1. Frameworks & Backend Technologies

- FastAPI Documentation: <https://fastapi.tiangolo.com/>
- SQLAlchemy Documentation: <https://www.sqlalchemy.org/>
- Uvicorn: <https://www.uvicorn.org/>
- Jinja2 Documentation: <https://jinja.palletsprojects.com/>

2. Modeling & Standards

- Unified Modeling Language (UML) Specification: <https://www.uml.org/>

3. AI Platforms / Chatbot Development

- OpenAI GPT API Documentation: <https://platform.openai.com/docs>